

ragent Tools Reference

A catalog of the tools available to ragent agents, organized by category. Each tool includes its name, description, use cases, parameter schema, the system instruction the model receives, and a worked example.

Scope: Tool names, schemas, and usage patterns (168 statically registered tools plus the dynamic `mcp_tool`). For TUI workflow see [docs/howtos/tutorial.md](#). For hiding/exposing tool families see [docs/howtos/tool-visibility.md](#). For team coordination see [docs/howtos/teams.md](#).

Category Index

#	Category	Count	Visibility Switch
1	File Operations	18	always on
2	Shell	4	always on
3	Search	1	always on
4	Web	3	always on
5	Browser Automation	1	browser
6	MasterFetch	6	masterfetch
7	Code Intelligence	10	codeindex
8	Memory	5	always on
9	Git	18	always on
10	GitHub	11	github
11	GitLab	19	gitlab
12	Office & PDF	8	office
13	Teams	20	teams
14	Sub-Agents	5	agents
15	Planning	2	plan
16	Spec Management	5	always on
17	Task Management	4	always on
18	Scheduling	5	always on
19	Initiatives	1	always on
20	MCP	1	always on
21	Skills	1	always on
22	Interactive	4	always on
23	Utility	1	always on
24	Finance	8	finance
25	Communications	2	always on
26	Plot	6	always on

Switches default off for `github`, `gitlab`, `teams`, `agents`, `plan`, `office`; the rest default on. See [docs/howtos/tool-visibility.md](#).

1. File Operations

Tool	Description
read	Read file contents with optional line range.
write	Create or overwrite a file.
create	Create a new file (preferred for new files).
edit	Replace one exact text occurrence in a file.
multi_edit	Apply multiple surgical edits atomically across files.
multiedit	Deprecated alias for multi_edit.
apply_patch	Apply a Codex-style patch (** Begin Patch).
patch	Apply a unified diff patch.
append_to_file	Append text to the end of a file.
rm	Delete a single file.
move_file	Move or rename a file/directory.
copy_file	Copy a file to a new location.
make_directory	Create a directory (mkdir -p).
file_info	Return metadata for a file or directory.
diff_files	Unified diff between two files or strings.
glob	Find files matching a glob pattern.
list	List directory contents in a tree.
update_file	Alias for write (overwrite existing file).

Use cases: scaffolding projects, editing source, comparing files.

Schema (edit): {"file_path":"string","old_string":"string","new_string":"string"}

System instruction: “Use edit for single surgical replacements. old_string must match exactly once. Use multi_edit for changes across multiple files.”

Fallback matching: when byte-for-byte matching fails, a fallback cascade retries with whitespace-flexible and indent-normalised matching. A whitespace run may only fold across a line boundary when both the needle run and the matched file run contain a newline, so the flexible lane can never join or split lines.

Example:

```
edit file_path="src/main.rs" old_string="fn main() {" new_string="fn main() -> Result<>> {"
```

2. Shell

Tool	Description
bash	Execute a shell command (7-layer security).
bash_reset	Reset persistent shell state.
bg	Manage background shell tasks (spawn, list, wait, cancel).
open	Open a file/folder/URL via the desktop handler.

Use cases: running builds, tests, git operations, long-running tasks.

Schema (bash): {"command":"string","timeout":"int?"}

System instruction: “For simple commands use bash with the `command` parameter immediately. Do not describe what you will run.”

Example:

```
bash command="cargo test -- --nocapture" timeout=600
```

3. Search

Tool	Description
grep	Search file contents for a regex pattern (ripgrep).

Use cases: finding TODOs, searching comments, non-symbol text patterns.

Schema: `{"pattern":"string","path":"string?","include":"string?"}`

System instruction: “Use `grep` for arbitrary text and pattern matching. Use `codeindex` tools for code symbol queries.”

Example:

```
grep pattern="TODO" path="src" include="*.rs"
```

4. Web

Tool	Description
webfetch	Fetch a URL via HTTP GET; HTML converted to text.
websearch	Web search returning titles, URLs, snippets.
http_request	Full HTTP method/headers/body control.

Use cases: fetching documentation, testing APIs, simple web scraping.

Example:

```
webfetch url="https://rust-lang.org"
http_request url="https://api.github.com/repos/rust-lang/rust" method="GET"
```

5. Browser Automation

Tool	Description
browser	Chrome DevTools Protocol automation (open, click, type, screenshot, etc.).

Use cases: interacting with dynamic web pages, form filling, screenshots.

Schema: {"action": "open|click|type|screenshot|...", "url": "string?"}

Example:

```
browser action="open" url="https://example.com"
browser action="screenshot"
```

6. MasterFetch

Tool	Description
mf_fetch	Rich URL/PDF extraction with envelope signals.
mf_search	Keyless multi-engine web search.
mf_crawl	Best-first same-domain crawl.
mf_cache_clear	Clear the content cache.
mf_screenshot	Capture a page as a screenshot.
mf_version	Return integration version info.

Use cases: extracting article content, searching multiple engines, crawling a domain.

mf_search runs DuckDuckGo, Brave, OpenAlex, and Wikipedia in parallel (plus optional LangSearch / Tavily / Perplexity / Exa / Serper engines when their keys are configured). An optional exclude_engines array removes named backends from the orchestrator *before* any request is dispatched, so excluded engines are never queried (unknown names are ignored); naming every configured engine returns an explicit “all engines excluded” result. An optional engine restricts the search to a single backend.

Example:

```
mf_fetch url="https://example.com" format="markdown"
mf_search query="rust async patterns" max_results=10
mf_search query="rust async patterns" exclude_engines=["openalex"]
```

7. Code Intelligence

Tool	Description
codeindex_search	Search symbols by name/keyword.
codeindex_symbols	Query symbols with kind/file filters.
codeindex_references	Find all references to a symbol.
codeindex_dependencies	Query file import/dependent edges.
codeindex_status	Show index status and statistics.
codeindex_reindex	Trigger a full re-index.
codeindex_explain	Explain a graph node and its edges.
codeindex_path	Shortest path between two symbols.
codeindex_communities	Community detection over the graph.
codeindex_godnodes	Top-N most-connected symbols.

Use cases: finding function definitions, tracing callers, dependency analysis, code graph exploration.

System instruction: “MUST use codeindex instead of grep for code symbol queries. Use grep only for arbitrary text patterns.”

Example:

```
codeindex_search query="parse_config" kind="function"
codeindex_references symbol="parse_config"
```

8. Memory

Tool	Description
memory_store	Store a structured memory (category, tags, confidence).
memory_recall	Full-text search of structured memories.
memory_forget	Delete memories by ID or filter.
conversation_search	Search current session conversation history.
session_search	Search across all past sessions.

Schema (memory_store): {"content": "string", "category": "fact|pattern|insight", "tags": [...], "confidence": 0.0-1.0}

Example:

```
memory_store content="We use anyhow::Result for errors" category="preference" confidence=0.9
memory_recall query="error handling"
```

9. Git

Tool	Description
git_add	Stage files for commit.
git_branch	List local and remote branches.
git_checkout	Switch branch or restore files.
git_cherry_pick	Apply changes from existing commits.
git_clone	Clone a repository.
git_commit	Create a commit from staged changes.
git_diff	Show working tree, staged, or commit diff.
git_fetch	Fetch from remote without merging.
git_log	Show commit history.
git_merge	Merge another branch.
git_pull	Fetch and integrate from remote.
git_push	Push branches and tags to remote.
git_remote	List, add, remove, update remotes.
git_reset	Unstage or reset to a commit.
git_show	Show commit, tag, or object details.
git_stash	Stash and unstash changes.

Tool	Description
git_status	Show working tree status.
git_tag	List, create, show, or delete tags.

Example:

```
git_status
git_add paths=["src/main.rs"]
git_commit message="Add greet subcommand"
git_push
```

10. GitHub

Tool	Description
github_list_issues	List issues (open/closed/all).
github_get_issue	Get full issue details.
github_create_issue	Create a new issue.
github_comment_issue	Comment on an issue.
github_close_issue	Close an issue.
github_list_prs	List pull requests.
github_get_pr	Get PR details.
github_create_pr	Create a new pull request.
github_merge_pr	Merge an open PR.
github_review_pr	Submit a PR review.
github_get_actions	List recent Actions workflow runs.

Requires /github login first. See docs/howtos/tutorial.md Section 7.

Example:

```
github_create_pr title="Add greet subcommand" base="main" body="Implements greet"
github_merge_pr number=42 method="squash"
```

11. GitLab

Tool	Description
gitlab_list_issues	List issues.
gitlab_get_issue	Get issue details.
gitlab_create_issue	Create an issue.
gitlab_comment_issue	Comment on an issue.
gitlab_close_issue	Close an issue.
gitlab_list_mrs	List merge requests.
gitlab_get_mr	Get MR details.

Tool	Description
gitlab_create_mr	Create a merge request.
gitlab_merge_mr	Merge an MR.
gitlab_approve_mr	Approve an MR.
gitlab_list_pipelines	List pipelines.
gitlab_get_pipeline	Get pipeline details.
gitlab_list_jobs	List jobs in a pipeline.
gitlab_get_job	Get job details.
gitlab_get_job_log	Get job log output.
gitlab_retry_job	Retry a failed job.
gitlab_cancel_job	Cancel a running job.
gitlab_retry_pipeline	Retry a pipeline.
gitlab_cancel_pipeline	Cancel a pipeline.

Requires /gitlab setup authentication.

12. Office & PDF

Tool	Description
office_read	Read DOCX/XLSX/PPTX content.
office_write	Write Office documents.
office_info	Return Office document metadata.
libre_read	Read ODT/ODS/ODP content.
libre_write	Write LibreOffice documents.
libre_info	Return LibreOffice document metadata.
pdf_read	Extract text from a PDF.
pdf_write	Write a PDF document.

Example:

```
office_read path="report.docx"
pdf_read path="spec.pdf"
```

13. Teams

Tool	Description
team_create	Create a named team from a blueprint.
team_spawn	Spawn a teammate for a scoped task.
team_message	Direct message a teammate or lead.
team_broadcast	Message all active teammates.
team_read_messages	Check mailbox for unread messages.
team_status	Team and task status summary.

Tool	Description
<code>team_task_list</code>	List all team tasks.
<code>team_task_create</code>	Lead adds a shared task.
<code>team_task_claim</code>	Teammate claims the next available task.
<code>team_task_complete</code>	Mark a claimed task done.
<code>team_assign_task</code>	Lead assigns a task to a teammate.
<code>team_submit_plan</code>	Teammate submits a plan to the lead.
<code>team_approve_plan</code>	Lead approves/rejects a plan.
<code>team_wait</code>	Block until teammates finish.
<code>team_idle</code>	Teammate signals no more work.
<code>team_shutdown_teammate</code>	Lead requests teammate shutdown.
<code>team_shutdown_ack</code>	Teammate acks shutdown and exits.
<code>team_cleanup</code>	Lead tears down the team.
<code>team_memory_read</code>	Read team memory bucket.
<code>team_memory_write</code>	Write team memory bucket.

See docs/howtos/teams.md for the full team manual.

14. Sub-Agents

Tool	Description
<code>new_agent</code>	Spawn a sub-agent (blocking or background).
<code>cancel_agent</code>	Cancel a running background sub-agent.
<code>list_agents</code>	List sub-agent tasks for the session.
<code>wait_agents</code>	Block until background tasks complete.
<code>agent_complete</code>	Terminal signal: the autonomous task is done.

System instruction: “Prefer sub-agents over doing work yourself. Use `background: true` when spawning more than one.”

Example:

```
new_agent agent="explore" task="Find all usages of EventBus in src/" background=true
wait_agents
```

15. Planning

Tool	Description
<code>plan_enter</code>	Delegate to the plan agent for read-only analysis.
<code>plan_exit</code>	Exit plan mode.

16. Spec Management

Tool	Description
<code>spec_read</code>	Read a specification by ID.
<code>spec_list</code>	List all specifications.
<code>spec_search</code>	Search specifications by keyword.
<code>spec_task_update</code>	Update a task's status within a spec.
<code>spec_coverage</code>	Generate a requirement coverage report.

See docs/howtos/spec.md for the full spec workflow.

17. Task Management

Tool	Description
<code>task_create</code>	Create a session-scoped task.
<code>task_update</code>	Update task status, subject, or dependencies.
<code>task_get</code>	Retrieve a single task by ID.
<code>task_list</code>	List all session tasks.

Example:

```
task_create subject="Fix config loader" description="Reproduce panic and fix root cause"
task_update task_id="task-001" status="in_progress"
```

18. Scheduling

Tool	Description
<code>cron_add</code>	Create a scheduled agent run.
<code>cron_remove</code>	Delete a scheduled event.
<code>cron_list</code>	List all scheduled events.
<code>cron_enable</code>	Enable a scheduled event.
<code>cron_disable</code>	Disable a scheduled event.

```
Schema(cron_add): {"id":"string","agent":"general","schedule":"every 1h","prompt":"string"}
```

19. Initiatives

Tool	Description
initiative	Manage durable initiatives with milestones.

Actions: create, read, update, checkpoint, list, close.

20. MCP

Tool	Description
mcp_tool	Bridge to external Model Context Protocol servers.

Tools are dynamically named `mcp_<server>_<tool>` at runtime after discovery via `/mcp discover`.

21. Skills

Tool	Description
skill_manage	List, read, load, or reload skill packs.

Actions: list, read, load, reload.

22. Interactive

Tool	Description
ask_user	Ask the user a question (with optional choices).
think	Record a short reasoning note.
calculator	Evaluate a mathematical expression.
get_env	Read environment variable values.

Example:

```
ask_user question="Which build profile?" options=["Debug","Release"]
calculator expression="2 ** 32"
get_env name="ANTHROPIC_API_KEY"
```

23. Utility

Tool	Description
<code>model_info</code>	Report the active provider/model, capabilities, context window, and cost tier.

24. Finance

Tool	Description
<code>stock_quote</code>	Latest stock quote (price, volume, change).
<code>stock_history</code>	Historical OHLCV bars.
<code>stock_fundamentals</code>	Market cap, P/E, EPS, dividend yield, sector.
<code>stock_search</code>	Search ticker symbols by company name.
<code>stock_options</code>	Options chain (calls and puts).
<code>stock_recommendations</code>	Analyst recommendation trends.
<code>currency_rate</code>	Current exchange rate between two currencies.
<code>currency_history</code>	Historical exchange-rate OHLCV bars.

See `docs/howtos/finance.md` for configuration and provider details.

Example:

```
stock_quote symbol="AAPL"
stock_history symbol="MSFT" interval="1d" period="3mo"
currency_rate base="USD" quote="EUR"
```

25. Communications

Tool	Description
<code>gmail</code>	Search, read, draft, and send Gmail messages.
<code>send_channel_message</code>	Post to Telegram or Discord channels.

See `docs/howtos/communications.md` for OAuth2 setup and channel config.

Example:

```
gmail action="search" query="from:ci@example.com is:unread"
send_channel_message action="send" message="Build passed" channel="telegram"
```

26. Plot

Scientific/terminal plotting rendered off-screen to a text canvas via `ratatui-plt`. Every tool returns the canvas as plain text and mirrors it into output metadata (`plot plain canvas`, `plot_ansi` ANSI-coloured

canvas); the TUI renders the coloured canvas inline in the message window. Canvas is clamped to 220x80 cells. All tools are read-only and always visible.

Tool	Description
<code>plot_line</code>	XY line chart from one or more named series of [x, y] points.
<code>plot_scatter</code>	XY scatter with dot markers.
<code>plot_bar</code>	Bar chart from categories + datasets; stacked and horizontal modes.
<code>plot_histogram</code>	Histogram over a sample array; count/density/probability normalisation.
<code>plot_pie</code>	Pie (or donut) chart from labelled slices; auto 8-colour palette cycling.
<code>plot_heatmap</code>	2D grid heatmap; viridis, plasma, inferno, magma, coolwarm colormaps.

Common optional arguments: `title`, `x_label`, `y_label`, `width` (default 80, max 220), `height` (default 20, max 80). Colours accept named colors (red, cyan, lightgreen, ...) or #rrggbb hex strings. String-encoded series/categories/data/slices/grid payloads are coerced automatically; malformed input returns an error output rather than crashing.

Example:

```
plot_line title="Latency" series=[{"name":"p50","data":[[1,12],[2,11],[3,9]],"color":"cyan"},{"na
plot_bar categories=["openalex","wikipedia"] datasets=[{"name":"hits","data":[14,22]}]
plot_histogram data=[3,5,5,7,9,9,9,11] bins=4
plot_pie slices=[{"label":"rust","value":71},{"label":"other","value":29}]
plot_heatmap grid={"values":[[0,1],[2,3]]} colormap="viridis"
```

Related Documents

Document	Covers
<code>docs/howtos/tutorial.md</code>	End-to-end TUI workflow tutorial
<code>docs/howtos/tool-visibility.md</code>	Hiding and exposing tool families
<code>docs/howtos/teams.md</code>	Multi-agent team coordination
<code>docs/howtos/communications.md</code>	Gmail and messaging channel tools
<code>docs/howtos/finance.md</code>	Stock and currency tools
<code>docs/howtos/spec.md</code>	Spec management and SDD workflow
<code>docs/howtos/reverse.md</code>	Repository reverse-engineering
<code>docs/howtos/research.md</code>	Research system and report synthesis
<code>docs/howtos/custom-agents.md</code>	Custom agent profiles and OASF schema